# Data description

## Data description I

The demonstration dataset consists of two comma-separated files under `data/fixtures/`, described by the machine-readable descriptor `data/example_descriptor.json`:

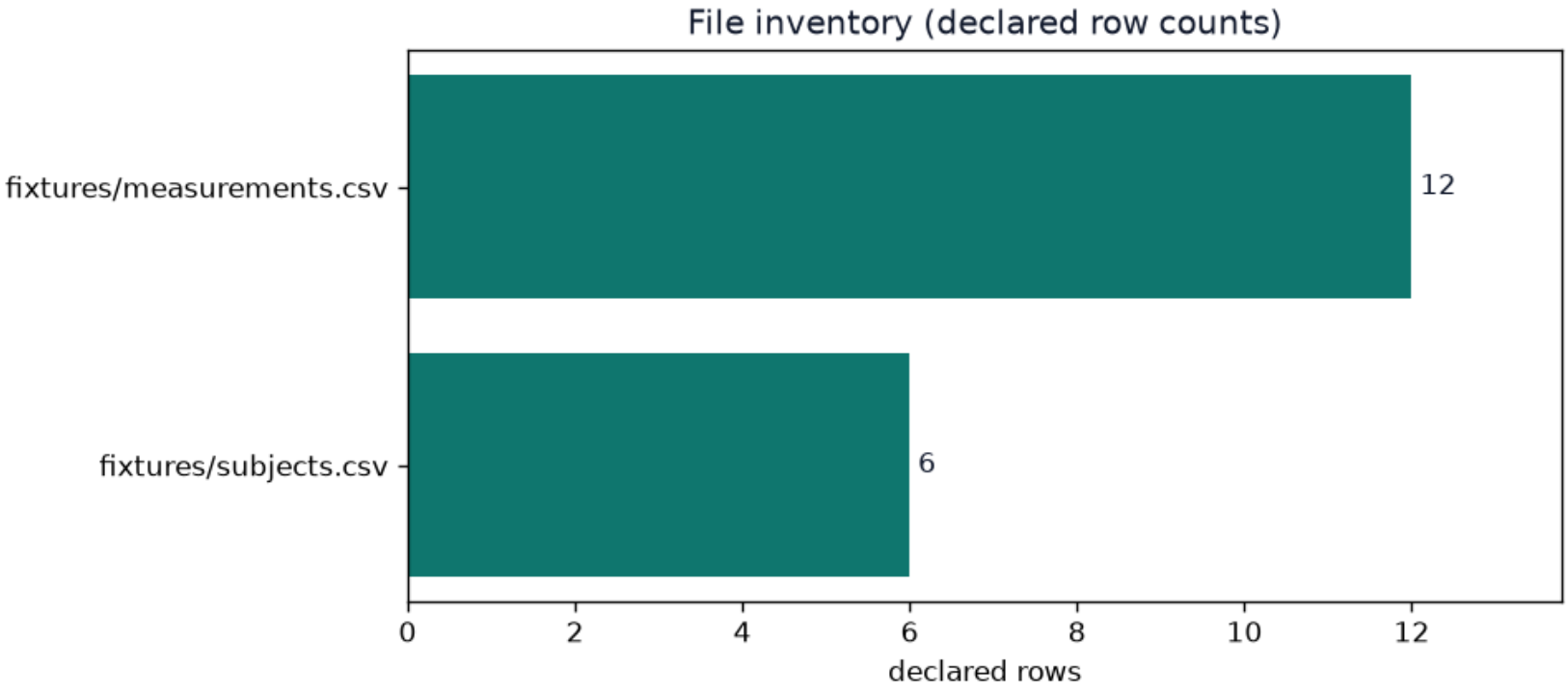
# Data description I

- `fixtures/measurements.csv` — one row per synthetic sample, with a bounded measurement, an assignment group, a capture instrument, and a collection date.
- `fixtures/subjects.csv` — one row per synthetic subject, with an enrollment site and date. Each measurement references a subject through the `subject_id` foreign key.

## File inventory I

The descriptor declares each file's media type, sha256 checksum, and row count. fig. 1 shows the declared row counts, read directly from the descriptor by `file_inventory_rows()` in `src/data_descriptor/figures.py` and plotted by the thin analysis script `scripts/generate_figures.py`.

# File inventory I



## File inventory I

Figure 1: File inventory: declared row counts per file, from `file_inventory_rows()`. The measurement table is the larger of the two files; the subject table is its dimension companion. Both are declared as `text/csv`. The bar values are the row counts recorded in the descriptor, not re-derived here — the quality gate (sec. 6) is where declared and actual counts are reconciled.

## File inventory I

Running the script regenerates the figures under `output/figures/`; the prose below and in later sections describes what those artifacts show. Because the figures are produced from the descriptor and the fixture bytes, they cannot drift from the data without the tests and the quality gate noticing.

## Release boundary I

The descriptor is deliberately a *metadata* object. For a real dataset, the descriptor and its checksums can be published even when the underlying bytes are access-controlled: the checksums let a downstream user verify integrity once they obtain the files through the appropriate channel. In this template the fixture bytes are public and small, so they are shipped alongside the descriptor and used to demonstrate end-to-end verification.